

Lab experiments

Experiment -11 : Map Coloring using CSP (Constraint Satisfaction Problem)

Aim

To develop a Python program that solves the **Map Coloring Problem** using the **Constraint Satisfaction Problem (CSP)** approach, ensuring that no two neighboring regions have the same color.

Objective

- To understand the concept of CSP.
- To assign colors to map regions without violating constraints.
- To implement a simple backtracking algorithm in Python.

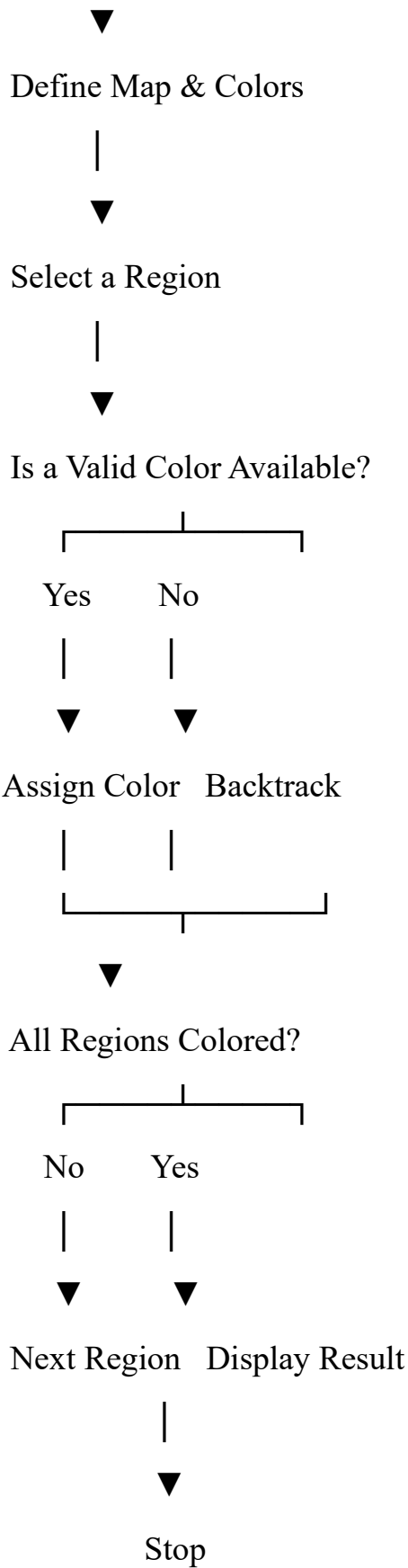
Algorithm

1. Define the map and its neighboring regions.
2. Select one region at a time.
3. Assign a color that is different from all adjacent regions.
4. If no valid color is available, backtrack and try another color.
5. Repeat until all regions are colored.
6. Display the final color assignment.

Flowchart

Start

|



Python Code

```
# Map Coloring using CSP (Backtracking)
```

```
colors = ["Red", "Green", "Blue"]
```

```
graph = {
```

```
    "A": ["B", "C"],
```

```
    "B": ["A", "C", "D"],
```

```
    "C": ["A", "B", "D"],
```

```
    "D": ["B", "C"]
```

```
}
```

```
color_map = {}
```

```
def is_safe(region, color):
```

```
    for neighbor in graph[region]:
```

```
        if color_map.get(neighbor) == color:
```

```
            return False
```

```
    return True
```

```
def solve(regions, index):
```

```
    if index == len(regions):
```

```
        return True
```

```
    region = regions[index]
```

```
    for color in colors:
```

```
        if is_safe(region, color):
```

```
            color_map[region] = color
```

```
            if solve(regions, index + 1):
```

```
                return True
```

```
            color_map[region] = None
```

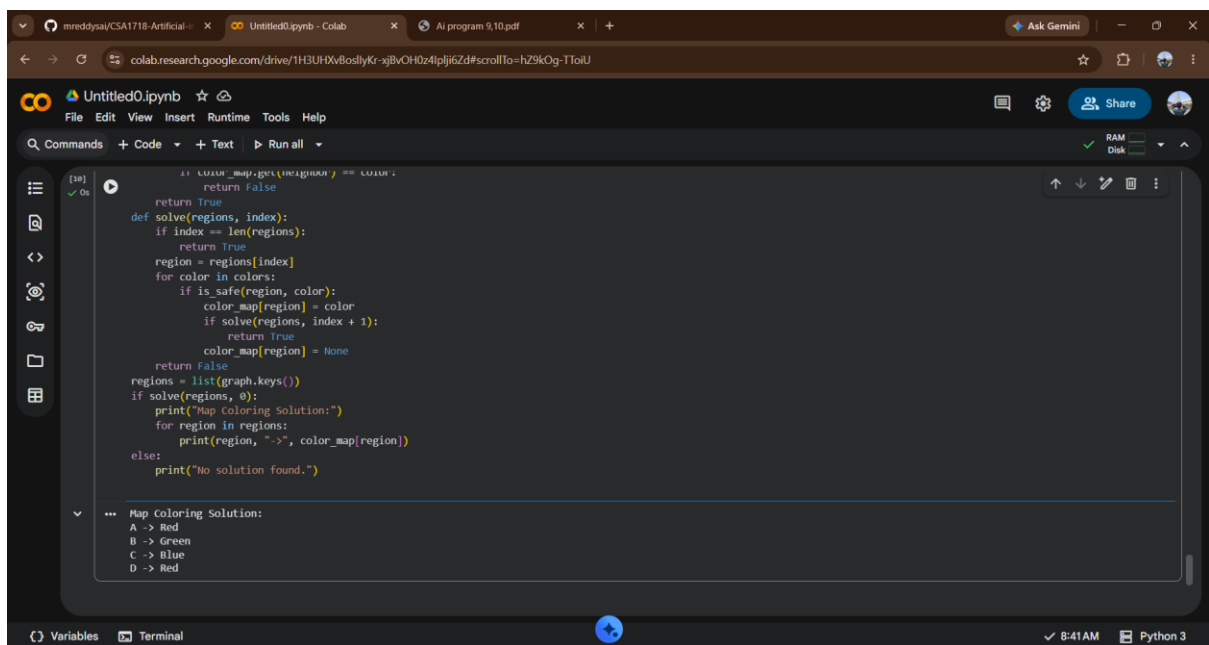
```
    return False
```

```

regions = list(graph.keys())
if solve(regions, 0):
    print("Map Coloring Solution:")
    for region in regions:
        print(region, "->", color_map[region])
else:
    print("No solution found.")

```

Output



The screenshot shows a Google Colab notebook interface. The code cell contains a recursive backtracking algorithm for map coloring. The output of the code is displayed below the cell, showing the solution for a map with four regions: A, B, C, and D.

```

1. color_map.get(neighbour) == color:
    return True
    return False
def solve(regions, index):
    if index == len(regions):
        return True
    region = regions[index]
    for color in colors:
        if is_safe(region, color):
            color_map[region] = color
            if solve(regions, index + 1):
                return True
            color_map[region] = None
    return False
regions = list(graph.keys())
if solve(regions, 0):
    print("Map coloring solution:")
    for region in regions:
        print(region, "->", color_map[region])
else:
    print("No solution found.")

```

Map Coloring Solution:

- A -> Red
- B -> Green
- C -> Blue
- D -> Red

Result

The map was successfully colored using the CSP backtracking algorithm, ensuring that no neighboring regions share the same color.

Conclusion

The Map Coloring Problem was solved using the CSP approach with backtracking. The program efficiently assigns valid colors while satisfying all constraints, making it a simple example of constraint-based problem solving.

Experiment-12 : Tic-Tac-Toe Game using Python

Aim

To create a simple **Tic-Tac-Toe game** in Python where two players can play by taking turns until one player wins or the game ends in a draw.

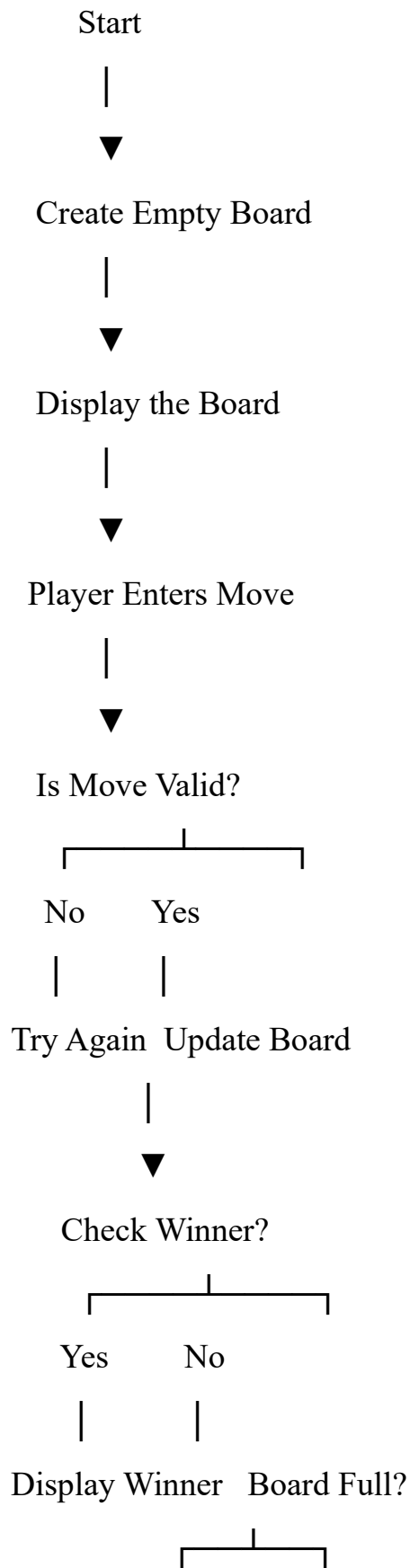
Objective

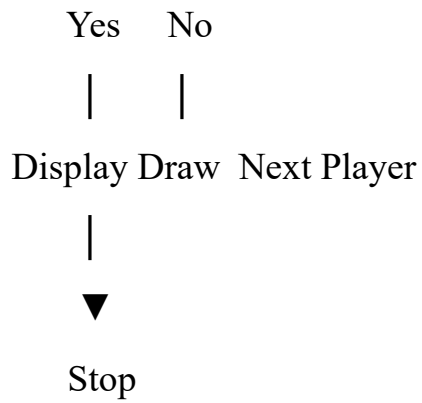
- To understand game logic in Python.
- To implement a turn-based game using lists and loops.
- To check winning and draw conditions.

Algorithm

1. Create a 3×3 game board.
2. Display the empty board.
3. Let Player **X** and Player **O** take turns entering positions.
4. Update the board after each valid move.
5. Check if any player has won.
6. If all cells are filled without a winner, declare a draw.
7. End the game.

Flowchart





Python Code

```
board = [" "]*9
```

```
def show():
```

```
    print()
```

```
    print(board[0], "|", board[1], "|", board[2])
```

```
    print("--+---+--")
```

```
    print(board[3], "|", board[4], "|", board[5])
```

```
    print("--+---+--")
```

```
    print(board[6], "|", board[7], "|", board[8])
```

```
    print()
```

```
def win(p):
```

```
    w = [(0,1,2),(3,4,5),(6,7,8),
```

```
          (0,3,6),(1,4,7),(2,5,8),
```

```
          (0,4,8),(2,4,6)]
```

```
    return any(board[a]==board[b]==board[c]==p for a,b,c in w)
```

```

player = "X"

for _ in range(9):
    show()
    pos = int(input(f'Player {player}, enter position (1-9): ')) - 1

    if board[pos] == " ":
        board[pos] = player

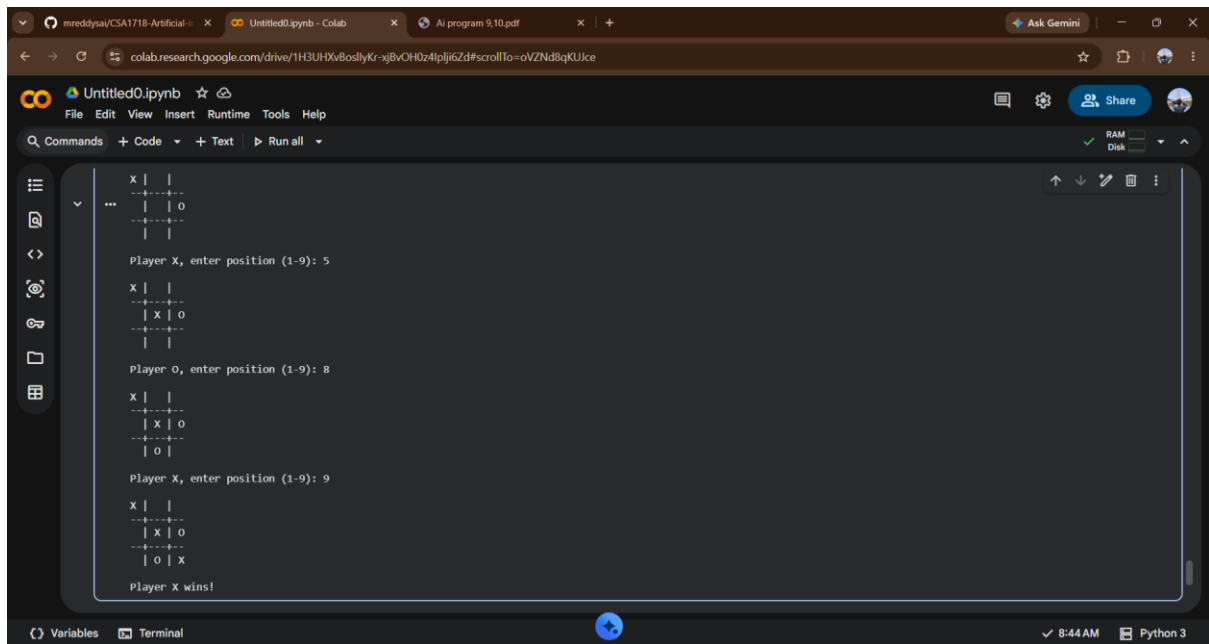
        if win(player):
            show()
            print("Player", player, "wins!")
            break

        player = "O" if player == "X" else "X"
    else:
        print("Position already taken!")

else:
    show()
    print("Match Draw!")

```

Output



```
x | |
--|---
| | o
--|---
| |

Player X, enter position (1-9): 5

x | |
--|---
| x | o
--|---
| |

Player O, enter position (1-9): 8

x | |
--|---
| x | o
--|---
| o |

Player X, enter position (1-9): 9

x | |
--|---
| x | o
--|---
| o | x

Player X wins!
```

Player X wins!

Result

The Tic-Tac-Toe game was successfully implemented in Python, allowing two players to play alternately and correctly identifying the winner or a draw.

Conclusion

The experiment demonstrates how Python can be used to build a simple interactive game. It improves understanding of loops, conditions, lists, functions, and basic game development concepts in an easy and practical way.

Experiment-13: Minimax Algorithm for Game Decision Making

Aim

To implement the **Minimax Algorithm** in Python to find the best move in a two-player game by maximizing the player's score and minimizing the opponent's score.

Objective

- To understand the working of the Minimax algorithm.
- To simulate decision-making in a two-player game.
- To find the optimal move using recursion.

Algorithm

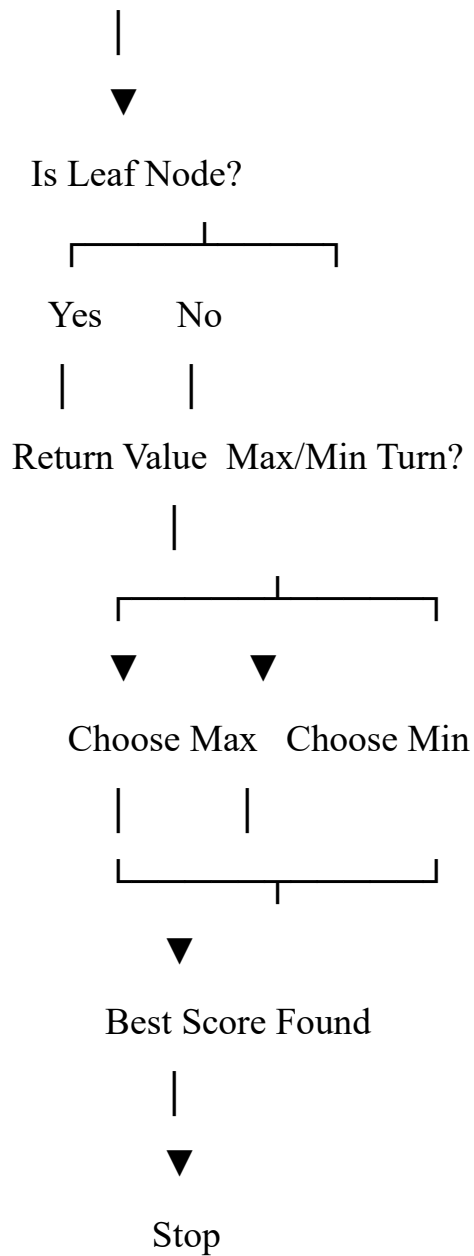
1. Define the game values (leaf nodes).
2. Start from the root node.
3. If it is the **Max** player's turn, choose the maximum value.
4. If it is the **Min** player's turn, choose the minimum value.
5. Continue recursively until the leaf nodes are reached.
6. Display the best possible score.

Flowchart

Start



Define Game Values



Python Code

Minimax Algorithm in Python

```
import math

values = [3, 5, 2, 9]

def minimax(depth, index, isMax):
    if depth == 2:
```

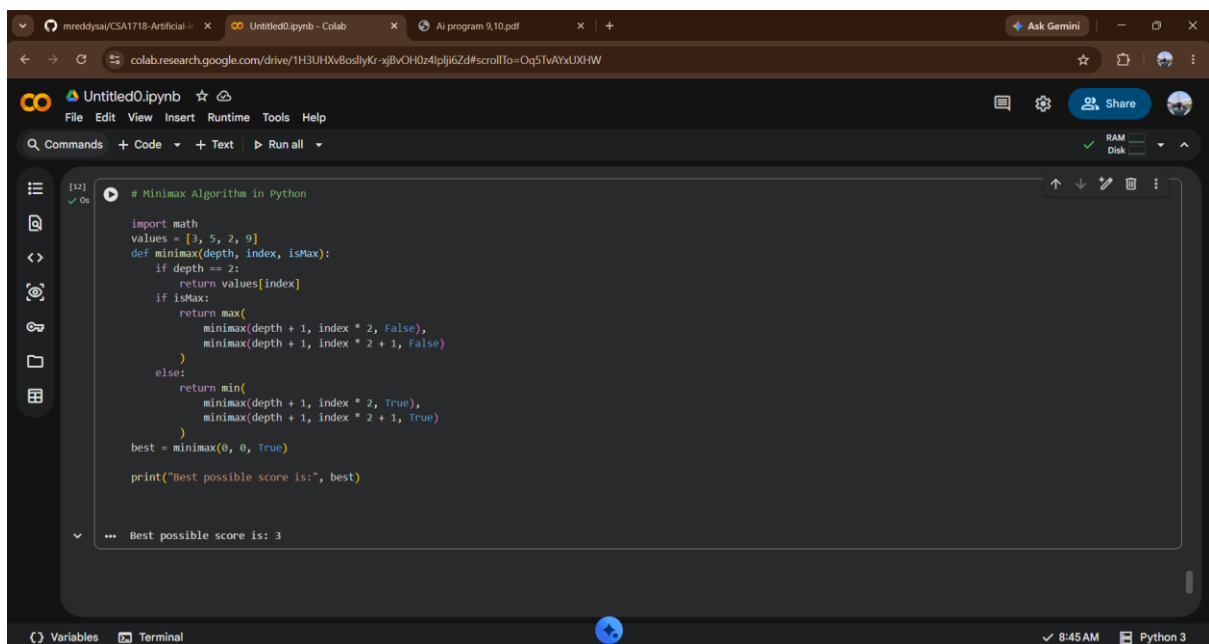
```

        return values[index]
    if isMax:
        return max(
            minimax(depth + 1, index * 2, False),
            minimax(depth + 1, index * 2 + 1, False)
        )
    else:
        return min(
            minimax(depth + 1, index * 2, True),
            minimax(depth + 1, index * 2 + 1, True)
        )
best = minimax(0, 0, True)

print("Best possible score is:", best)

```

Output



The screenshot shows a Google Colab notebook titled 'Untitled0.ipynb'. The code cell contains the following Python code:

```

# Minimax Algorithm in Python

import math
values = [3, 5, 2, 9]
def minimax(depth, index, isMax):
    if depth == 2:
        return values[index]
    if isMax:
        return max(
            minimax(depth + 1, index * 2, False),
            minimax(depth + 1, index * 2 + 1, False)
        )
    else:
        return min(
            minimax(depth + 1, index * 2, True),
            minimax(depth + 1, index * 2 + 1, True)
        )
best = minimax(0, 0, True)

print("Best possible score is:", best)

```

The output of the code is displayed below the code cell:

```

--- Best possible score is: 3

```

Best possible score is: 3

Result

The Minimax algorithm was successfully implemented, and it correctly selected the best possible score for the maximizing player.

Conclusion

The Minimax algorithm is an effective decision-making technique for two-player games. It helps the player choose the optimal move by considering both the player's and the opponent's possible actions. This experiment provides a simple understanding of game AI using Python.